Task 15

1. System overview

Smart File & Media manager is a Windows-based desktop Application built using Python with a PyQt6 graphical interface. Users may select folders and browse their files to review the name, type, size and location of each file selected. Search functionality will also allow users to search the list of files based on specific criteria such as keyword. Results may be filtered based on categories such as “images”, “audio” etc. Preview image functionality has been included allowing users to view images directly within the program. Audio playback has also been implemented providing users the ability to pause/resume audio playback. Finally, a CSV report option provides users with a detailed summary of the files processed including the path, file type and size.

The primary objective was to create a singular interface that includes multiple common file management functions. In addition, event driven programming techniques were applied to provide both multimedia capabilities and Background processing. Testing and version control methods were also utilized to assist in the process of developing this software solution.

2. Design and separation of concerns

Module	responsibilities
Main.py	Application entry point and initialization.
Main_window.py	PyQt6 user interface, file table, preview panel and controls.
Controller.py	user actions and operations related to the GUI and file-logic processing.
File_model.py	file records, scanning, filtering and CSV related operations.
Workers.py	Background processing and communication with the GUI.
Performance.py	benchmarking Sequential vs threaded performance comparison.
Tests/test_file_model.py	automated verification of non-GUI logic.

This architecture separates the presentation layer from the coordination layer and the data access layer instead of combining all functionality in a single window class. Therefore, testing the file logic without testing the entire GUI is simpler. The overall separation of concerns follows the model view controller pattern, although some of the final class names and signal-slot relationships should be verified against the submitted source code.

3. GUI design decisions

A simple design approach was chosen for creating the interface. Top level options include Open Folder, Start scan, Cancel scan and export CSV. Options to search and filter the results are positioned next to the table. Additionally, a media preview panel is located below the table so that users do not need to perform additional navigation to preview media. Background activities are visually represented via status text and a progress bar. Cancelling a scan is disabled until an active scan exists to prevent an illegal action. This design approach emphasizes usability and immediate feedback over visual appeal.

4. Concurrency, synchronization and performance

Processing a folder with potentially thousands of files requires numerous filesystem operations. Performing these operations on the same GUI event loop would likely freeze the GUI. As such, Background workers are used to execute these operations. Progress updates or notifications of completion are communicated from the worker to the PyQt6 interface using thread-safe qt signal/slot communications. Updates made to ui components should occur on the GUI thread; however, the worker notifies of completed tasks using signals sent via slots.

Results measured:

Measure	observations
Test folder	d:/music
Total files in Test folder	350
Number of workers used	4
Number of benchmark runs performed	3
Average time per Sequential run	0.0181 sec
Average time per threading run	0.0224 sec
Ratio of average times (threaded seq.)	0.81×

Predicted amdahl upper bound (p=0.8,N=4)	2.50×

Speed-up ratio = 0.0181 / 0.0224 ~ 0.81. Although threading resulted in slightly longer run times in this case due to the very low Number of total items being scanned and thus greater overhead associated with task coordination versus potential speed-ups realized from executing the scans in parallel. According to amdahl's law, if we assume that approximately 80% of the workload could be executed in parallel then we could expect a maximum theoretical improvement factor of 1 / ((1 – 0.8) + 0.8 / 4) = 2.50×. This is purely theoretical, i.e., neither an actual measured value nor an estimated percentage of the workload which could be performed in parallel based upon actual observation or measurement. The greatest benefit to utilizing Background processing in this example lies in maintaining the responsiveness of the GUI during processing rather than demonstrating shorter processing times.

5. Manual and automated testing, correction

Manual testing

During the development ten tests have been run which had shown positive results: launching the app, scanning files, finding/searching for files, filtering files, playing audio files, exporting CSV files, checking if there is no folder to scan, cancelling the scan, displaying previews for images, and running the scanner again. A separate set of tests has also been performed on the Windows packaged executable on the development PC which resulted in passing eight tests: launch, scan, find/search/filter, play MP3, display image preview, export CSV, cancel/open.

Automated testing

In approximately 0.043 seconds six unittest cases have passed: test_cancelled_scan, test_category, test_filter, test_missing_folder, test_scan_and_csv, and test_sizes_and_boundary. Automatic testing focuses primarily on the reusable file logic. The multimedia rendering and interaction that will occur within the GUI cannot be tested automatically due to their dependence upon the Windows environment for both multimedia and GUI functionality.
To run the suite use: py –m unittest discover -s ./tests –p “test_file_model.py” –v

Development Corrections

When a test directory name conflicts with Python's default test naming convention, it caused a problem to find the appropriate test discovery. The directory was named “tests” instead of “test”, and an explicit discovery path was used. Environment/package issues that arose while developing were corrected via environment/package checks. To provide additional evidence for corrective action include a copy of the screen shot taken at time stamp X in DEBUG_LOG.md or date-stamped version of the excerpt from DEBUG_LOG.md and evidence of a successful retest.

6. Git/source control Workflow

https://github.com/mrmanav111/smart-file-media-manager is the GitHub project. The feature/testing/documentatioin branch was created, committed (996ea8f) and pushed into main through Pull Request #1 (the merge commit being 2de7538). At this point the local working tree was synchronized with the remote. A controlled conflict was created by editing git_conflict_demo.txt differently on main and feature/conflict-demo, and then resolved by creating the merge commit ee14932. As per previous request the distribution guide and PyInstaller specification were committed as 6032498. The Git history allows for reviewability and shows the ability to perform branch, commit, pull, push and merge operations. The controlled conflict was an educational tool and was not an error in the product itself.

7. Distributing the Application

Using PyInstaller 6.22.3 the application was converted into a single Windows folder distribution utilizing Python 3.13.0. The resultant executable can be found in dist/SmartFileMediaManager/SmartFileMediaManager.exe and must remain adjacent to the _internal directory. A zip archive containing the distribution folder is included as SmartFileMediaManager_Windows.zip. The build specifications and DISTRIBUTION.md were maintained within Git, whereas the compiled archive is a local deliverable. Successful builds and eight local executable tests were documented. It is recommended that testing occurs on a second Windows machine that does not contain Python installation; however, such testing has not currently been demonstrated.

8. Evaluating Critically and Identifying Areas of Future Improvement
Positive aspects:

The primary workflow utilized is easy to follow; the application performs multiple file management tasks; the background scan is non-blocking for the user interface; automated testing covers critical aspects of the file logic; and a reproducible Windows build exists. The design decisions made above reflect good separation of concerns related to presentation and processing. Negative aspects: Performance metrics represent only a small sample size of 350-file folders and do not represent large or network mounted directories. Due to performance differences measured between threading and process based approaches performance claims should be conservative. Additionally, multimedia support is dependent upon codec availability in addition to the host environment. Distribution has only been verified on the development computer. Further testing will need to utilize automated GUI testing in place of recording observed behavior during manual testing. Potential Next Steps:Compare performance across larger test folders/file types
Add regression testing and automated GUI testing
Verify the installer works properly on another Windows computer
Enhance error reporting and increase accessibility
Consider batching file table updates to decrease UI work when performing large scans
Evaluate Process-Based Approach For CPU-Intensive Analysis

Technical references

Python Software Foundation. unittest — Unit testing framework. https://docs.python.org/3/library/unittest.html

Qt Company. Signals & Slots (Qt 6). https://doc.qt.io/qt-6/signalsandslots.html

Qt Company. Threading Basics (Qt 6). https://doc.qt.io/qt-6/thread-basics.html

PyInstaller. Using PyInstaller. https://pyinstaller.org/en/stable/usage.html

Git. git-merge documentation. https://git-scm.com/docs/git-merge